# Software Requirements Engineering

## Complete Lab Exam Notes

Lab 1 Failure Analysis | Context & System Diagrams | Kano Model | Elicitation Techniques
Use Case Modeling | DFD | SRS | Activity & State Diagrams

**How to use these notes:** Read Part 0 first (exam strategy), then each lab section. Every section follows the same pattern: **concept in simple words -> what the lab asked -> model answer -> exam tips**. Items marked **[EXTRA]** are small additions beyond the manuals. The running example is the **University Course Registration System**, which appears in almost every lab, so learn it once and reuse it.

| Part | Topic | Key thing to remember |
|---|---|---|
| 1 | Requirements Engineering and failures (Lab 1) | Poor requirements -> misunderstanding -> rework -> cost/time -> failure |
| 2 | Context and System Diagrams | Context = 1 box + external entities; System = internal modules |
| 3 | Requirement Sources and Kano Model | Pair of answers -> evaluation matrix -> A/O/M/I/R/Q |
| 4 | Elicitation Techniques (Lab 5) | Evidence-based discovery, not guessing features |
| 5 | Use Case Modeling (+ SAEMS) | Actors outside, goals inside, include = mandatory, extend = conditional |
| 6 | Data Flow Diagram (Level 1) | Processes, stores, entities, flows are DATA not actions |
| 7 | SRS (ISO/IEC/IEEE 29148) | Shall statements, measurable, verifiable, traceable |
| 8 | Activity and State Diagrams | Activity = workflow; State = life of one object |
| 9 | Final cheat sheet and likely questions | Last-hour revision |

# Part 0: Exam Strategy and Common Vocabulary

## 0.1 How these labs connect

All the labs describe the **same learning chain** of Requirements Engineering (RE): find requirements (*elicitation, sources, Kano*), understand and organise them (*analysis, models*), write them down properly (*specification, SRS*), check them (*validation*), and rank them (*prioritisation*). The diagrams are different *views* of the same requirements.

| View | Diagram | Question it answers |
|---|---|---|
| Boundary view | Context diagram | What is inside/outside the system and what flows across the boundary? |
| Internal structure | System diagram | What major modules does the system have? |
| User goals | Use case diagram | Who uses the system and for what goal? |
| Data movement | DFD | How does data move between processes, stores and entities? |
| Workflow | Activity diagram | What are the steps, decisions and loops of a process? |
| Life cycle | State machine diagram | What states does ONE object go through and what events change them? |

## 0.2 Key definitions to be able to write instantly

- **Requirements Engineering:** the systematic process of eliciting, analysing, specifying, validating, documenting and managing software requirements.
- **Requirement:** a statement of what a system must do (functional) or how well/under what constraints it must do it (non-functional).
- **Stakeholder:** any person/group affected by or influencing the system (students, teachers, registrar, IT admin, management, etc.).
- **Functional Requirement (FR):** a specific behaviour or function, e.g. "The system shall allow a student to drop a course."
- **Non-Functional Requirement (NFR):** a quality or constraint (performance, security, usability, availability, reliability, scalability, portability, maintainability), e.g. "95% of searches within 2 seconds."
- **Elicitation:** discovering requirements from people, documents, systems. **Analysis:** checking conflicts/gaps. **Specification:** writing clearly. **Validation:** confirming they are right. **Management:** handling changes and traceability.

## 0.3 Exam strategy

- **Read the scenario twice** and underline nouns (actors, entities, data stores) and verbs (goals, processes, functions).
- For any diagram question: state **assumptions** first in one line, then draw, then give a 2-line explanation. Examiners give marks for justification.
- For requirement-writing questions use the **shall** format with a **number, condition and measurable threshold**.
- For classification questions (FR vs NFR, Kano, MoSCoW) always add a **one-line reason**.
- Neat labelling matters: label every arrow, name every module, number every process (1.0, 2.0 ...).

# Part 1: Requirements Engineering and Project Failure Analysis (Lab 1)

## 1.1 Why RE matters

Requirements engineering is the foundation of software projects. Poor requirements cause **misunderstanding, rework, delays, increased cost, poor quality, user dissatisfaction and project failure**.

> **Key lesson (memorise):** Poor requirements -> misunderstanding -> rework -> increased cost/time -> poor-quality system -> possible project failure.

## 1.2 Is "We need a simple system where students can register for courses" enough?

**No.** It is vague. The scenario shows stakeholder statements: Student (register online; system should be fast), Teacher (see who registered for my course), Registrar (no registration without prerequisite completion), IT Admin (available during registration week). These are needs, but they are incomplete, unmeasured and from different viewpoints, so they must be **elicited, analysed, specified, validated and managed** before development.

## 1.3 Types of requirement problems

| Problem | Meaning | Example |
|---|---|---|
| Ambiguous | More than one interpretation | "User-friendly", "fast" |
| Incomplete | Missing information/conditions | "Students can register for courses" (no rules, no failure case) |
| Not measurable / not testable | No number or criterion to verify | "The system should be secure" |
| Conflicting | Two requirements contradict | "Register whenever they want" vs registrar's registration period rule |
| Missing | Needed requirement never stated | No password reset, no waiting list |
| Changing | Requirement keeps changing during development | Rules changed by registrar late in the project |

## 1.4 Activity 1: Identify problems in R1-R6

| ID | Requirement | Problem |
|---|---|---|
| R1 | The system should be user-friendly. | Ambiguous, not measurable (no definition of friendly). |
| R2 | Students can register for courses. | Incomplete: no rules (prerequisites, seats, limits), no failure behaviour, no authentication. |
| R3 | The system should be fast. | Ambiguous, not measurable: no time, load, percentile. |
| R4 | Register for up to 6 courses. | Clear and testable but incomplete: per semester? credit hours? what message at limit? Possible conflict with credit-hour rules. |
| R5 | The system should be secure. | Ambiguous, too broad: what threats? authentication, encryption, roles? |
| R6 | Register whenever they want. | Conflicts with registration windows/deadlines and system availability; unrealistic, untestable. |

## 1.5 Activity 2: Rewriting poor requirements (make them clear, measurable, testable)

| Poor | Improved |
|---|---|
| The system should be fast. | The system shall display the course registration page within 2 seconds for at least 95% of requests under 500 concurrent users. |
| The system should be secure. | The system shall require authenticated login, encrypt passwords using salted hashing, and lock an account for 15 minutes after 3 consecutive failed login attempts. |
| The system should be easy to use. | A first-time student shall be able to register for a course within 3 minutes without external help, as measured in usability testing with at least 10 students (success rate at least 90%). |

**Formula for a good rewrite:** The system shall + [action] + [condition/scope] + [measurable value: time / number / percentage / load] + [how verified].

## 1.6 Activity 3: Elicitation - who gives what

| Stakeholder | Provides |
|---|---|
| Student | Needs and problems about course registration |
| Teacher | Requirements about course and student information |
| Registrar | Academic and registration rules |
| IT Administrator | Operational, performance, security and availability concerns |

**Questions a requirements engineer should ask:** What does the system need to do? What problems do you face now? What information do you need? What rules/restrictions must be enforced? What should happen on error? What are the security concerns? What performance do you expect?

## 1.7 Activity 4: Classify FR vs NFR

| Requirement | Class | Reason |
|---|---|---|
| Student can register for a course | FR | A function/behaviour |
| System shall respond within 2 seconds | NFR (performance) | Quality/time measure |
| Only authenticated students can register | NFR (security) / access-control FR | Restriction on who may use the function; commonly security NFR |
| Student can drop a course | FR | A function |
| Support 5,000 concurrent users | NFR (scalability/performance) | Capacity quality |

## 1.8 Activity 5: Missing requirements (existing: login, view courses, register, drop)

| Missing requirement | Why important |
|---|---|
| Prerequisite checking | Stops ineligible registration; academic rule |
| Seat availability / capacity check | Prevents over-enrolment |
| Registration confirmation / notification | Proof and clarity for student |
| Password reset / account lock | Security and access recovery |
| Drop deadline rule | Prevents late drops; academic policy |
| Timetable clash detection | Avoids invalid schedules |
| Maximum credit hours / course limit | Academic regulation |
| Teacher view of enrolled students; admin management of courses | Other stakeholders' needs |
| Error handling, audit logs, performance/availability targets | Quality, traceability |

## 1.9 Activity 6: Requirement validation

Validation checks each requirement for being **Clear, Complete, Testable**:

| Requirement | Clear? | Complete? | Testable? | Improvement |
|---|---|---|---|---|
| The system shall be fast. | No | No | No | Add time, percentile, load |
| The system shall allow course registration. | Yes (roughly) | No | Partly | Add actor, conditions (prerequisites, seats), outputs, errors |
| Allow a student to register for max 6 courses. | Yes | Partly | Yes | Specify per semester, message when exceeded, link to credit-hour rule |

## 1.10 Activity 7: MoSCoW prioritisation

**Must** have (essential), **Should** have (important but not vital), **Could** have (nice to have), **Won't** have (this time).

| Requirement | Priority | Justification |
|---|---|---|
| Student login | M | Without authentication nothing else is safe or personal |
| Course registration | M | Core purpose of the system |
| Email notification | S | Important for communication but registration works without it |
| Dark mode | C (or W) | Cosmetic preference |
| Prerequisite checking | M | Academic rule demanded by the registrar; wrong registrations otherwise |

## 1.11 Real-world failure case studies

For each case, be ready to answer: purpose, stakeholders, what went wrong, requirement problems, inadequate RE activities, and what should have been done.

| | Denver Airport Baggage System | FBI Virtual Case File (VCF) | London Ambulance Service CAD |
|---|---|---|---|
| Purpose | Fully automated baggage handling for the new Denver International Airport (1990s) | Replace FBI's paper/legacy case system with an electronic case-management system (2000-2005) | Computer-aided dispatch of ambulances in London (1992) |
| Stakeholders | Airport authority, airlines (esp. United), baggage handlers, contractors, passengers | FBI agents, analysts, management, contractor (SAIC), Congress | Ambulance crews, control-room staff, patients, management, developers |
| What went wrong | Airport opening delayed about 16 months, huge cost overruns; system unreliable (jams, lost bags); eventually abandoned | Delivered late and unusable; cancelled after hundreds of millions spent | System failed at launch; delayed dispatches; crews and staff overloaded |
| Requirement problems | Unrealistic scope; late and changing requirements (airlines changed needs after design began); poor feasibility analysis | Constantly changing requirements, vague/overly detailed specs, poor stakeholder involvement and management | Ignored crew/user input, unrealistic timeline, incomplete requirements (error handling, load), poor validation and testing |
| Weak RE activities | Feasibility, stakeholder analysis, change management | Elicitation, validation, requirements management | Elicitation (user involvement), validation, specification |
| Should have | Involve all stakeholders early, freeze scope, prototype, incremental delivery | Stable baseline, change control, regular stakeholder reviews | Involve crews, pilot/phased rollout, clearer requirements and testing |

**[EXTRA]** Figures differ by source; in an exam give a safe statement like "delayed by over a year and cost hundreds of millions beyond budget" rather than exact numbers.

## 1.12 Hospital Appointment Management System (independent task) - model answers

### Task 1: Stakeholders

| Stakeholder | Interest / need |
|---|---|
| Patient | Easily search doctors, view slots, book/cancel, get confirmation, privacy of data |
| Doctor | View appointments, update availability, cancel when necessary, avoid double booking |
| Administrator | Manage doctors/departments, view appointment records and reports |
| Hospital management | Efficient operations, reduced waiting time, compliance, reports |
| Receptionist / support staff | Assist patients with bookings, handle walk-ins |
| IT/support team | Security, maintenance, backups, availability |
| Regulators / legal | Patient data protection, record keeping |

### Task 2: 10 elicitation questions

- What steps do patients currently follow to book an appointment, and what problems occur?

- What information must a patient provide when creating an account?
- How do doctors currently publish and change their availability?
- What are the appointment slot durations, and can they vary by department?
- What should happen if a patient or doctor cancels, and how much notice is required?
- What confirmations/notifications (email, SMS) should be sent and when?
- Who is allowed to see appointment and patient information (roles and privacy)?
- What reports do administrators need from appointment records?
- How many users are expected at peak times and what response time is acceptable?
- What should happen on errors, e.g. double booking or system downtime?

### Task 3: 10 FRs and 5 NFRs (shall format)

| ID | Functional requirement |
|---|---|
| FR-01 | The system shall allow a patient to create an account using name, email, phone number and password. |
| FR-02 | The system shall allow a registered patient to log in using valid credentials. |
| FR-03 | The system shall allow a patient to search for doctors by name, department or specialty. |
| FR-04 | The system shall display available appointment slots for a selected doctor. |
| FR-05 | The system shall allow a patient to book an available slot. |
| FR-06 | The system shall prevent booking of a slot that is already booked. |
| FR-07 | The system shall allow a patient to cancel a booked appointment before the appointment time. |
| FR-08 | The system shall send a booking confirmation to the patient after successful booking. |
| FR-09 | The system shall allow a doctor to view appointments and update availability. |
| FR-10 | The system shall allow an administrator to manage doctors and departments and view appointment records. |

| ID | Non-functional requirement |
|---|---|
| NFR-01 (Performance) | The system shall display search results within 2 seconds for 95% of requests under 500 concurrent users. |
| NFR-02 (Security) | The system shall store passwords using salted hashing and encrypt patient data in transit using TLS. |
| NFR-03 (Availability) | The system shall be available 99.5% of the time monthly, excluding planned maintenance. |
| NFR-04 (Usability) | A new patient shall be able to book an appointment in under 3 minutes without assistance in 90% of usability tests. |
| NFR-05 (Scalability/Backup) | The system shall support at least 2,000 registered users and back up data daily. |

### Tasks 4-5: Poor requirements and improved versions

| Poor requirement | Problem type | Improved |
|---|---|---|
| The system should be fast. | Ambiguous / not measurable | Search results shall load within 2 s for 95% of requests under 500 concurrent users. |
| Patients can book appointments easily. | Ambiguous (easily) / incomplete | An authenticated patient shall be able to book an available slot in at most 4 steps and receive confirmation within 10 s. |
| The system should be secure. | Too broad / untestable | The system shall lock an account for 15 minutes after 3 failed logins and encrypt all patient data in transit. |
| Doctors can update availability. | Incomplete | A doctor shall be able to add or remove slots up to 30 days ahead; the system shall prevent removing a booked slot without cancelling the booking first. |
| Notifications should be sent soon. | Ambiguous (soon) | The system shall send a confirmation email within 1 minute of booking. |

### Task 6: Validation (five requirements)

| Requirement | Clear? | Complete? | Testable? | Improvement |
|---|---|---|---|---|
| FR-05 Book an available slot | Yes | Partly | Yes | Add confirmation and conflict handling |
| NFR-01 Search within 2 s | Yes | Yes | Yes | None |
| Doctors can update availability | Partly | No | Partly | Add time range and booked-slot rule |
| System should be secure | No | No | No | Replace with specific controls |
| FR-07 Cancel appointment | Yes | Partly | Yes | Add cancellation window and notification |

### Task 7: MoSCoW

| Requirement | Priority | Justification |
|---|---|---|
| Book appointment | M | Core purpose |
| Patient login / secure accounts | M | Protects medical data |
| Cancel appointment | M/S | Frees slots; needed for realistic operation |
| Email/SMS confirmation | S | Reduces no-shows but booking works without it |
| Online payment | C | Not mentioned in scenario; future extension |
| Dark mode | W | Not needed now |

### Task 8: Failure analysis

| Possible failure | Related RE activity | How it helps |
|---|---|---|
| Doctors' real availability rules not captured -> double bookings | Elicitation (interviews, observation) | Reveals how schedules work in practice |
| Patient data leaked | NFR specification and validation | Explicit, testable security requirements |
| System slow at peak hours | Measurable NFRs | Performance target defined and tested |
| Admin reports missing needed fields | Stakeholder analysis and workshops | All stakeholder needs identified early |
| Requirements changed repeatedly causing rework | Requirements management | Change control and traceability |

## 1.13 Assessment rubric (20 marks)

Stakeholder identification 2, elicitation questions 2, FRs 3, NFRs 2, poor and improved requirements 4, validation 2, prioritisation 2, failure analysis 2, presentation 1.

# Part 2: Context Diagram and System Diagram

## 2.1 Concepts

- **Context diagram:** external view. The whole system is ONE central box (process). External entities are outside; labelled, directional flows show what enters and leaves.
- **System diagram:** internal high-level view. Opens the box into 5-8 major modules and their relationships. NOT code, classes or database tables.
- **System boundary:** line separating what we build (inside) from the rest of the world (outside).
- **External entity:** person, organisation, device or other system that exchanges information with our system. An external system stays external even if one of our modules talks to it.

**Golden rules for the context diagram:** (1) exactly one central system, (2) show all major external entities, (3) every flow labelled with meaningful information (not "data"), (4) arrows have direction, (5) no internal modules, databases, screens or classes.

## 2.2 Scenario and requirements (Course Registration)

Students view courses, register, drop and view registered courses. Teachers view enrolled students. Academic/Examination Office manages course offerings and registration rules. System communicates with a University Authentication System. Extracted requirements: R1 authenticate, R2 view courses, R3 register, R4 drop, R5 view registered courses, R6 teacher views enrolled students, R7 office manages courses/rules, R8 external identity verification.

## 2.3 Worksheet: external entities and flows

| External entity | Why external? | Sends to system | Receives from system |
|---|---|---|---|
| Student | Person using the system | Login credentials, registration request, drop request, course view request | Authentication result, available courses, registration/drop confirmation, registered-course list |
| Teacher | Person outside the software | Enrollment information request | Enrolled-student information |
| Academic/Examination Office | External organisational role | Course offerings, registration rules | Course management / registration information |
| University Authentication System | Separate external software service | Authentication/verification result | Authentication (identity verification) request |

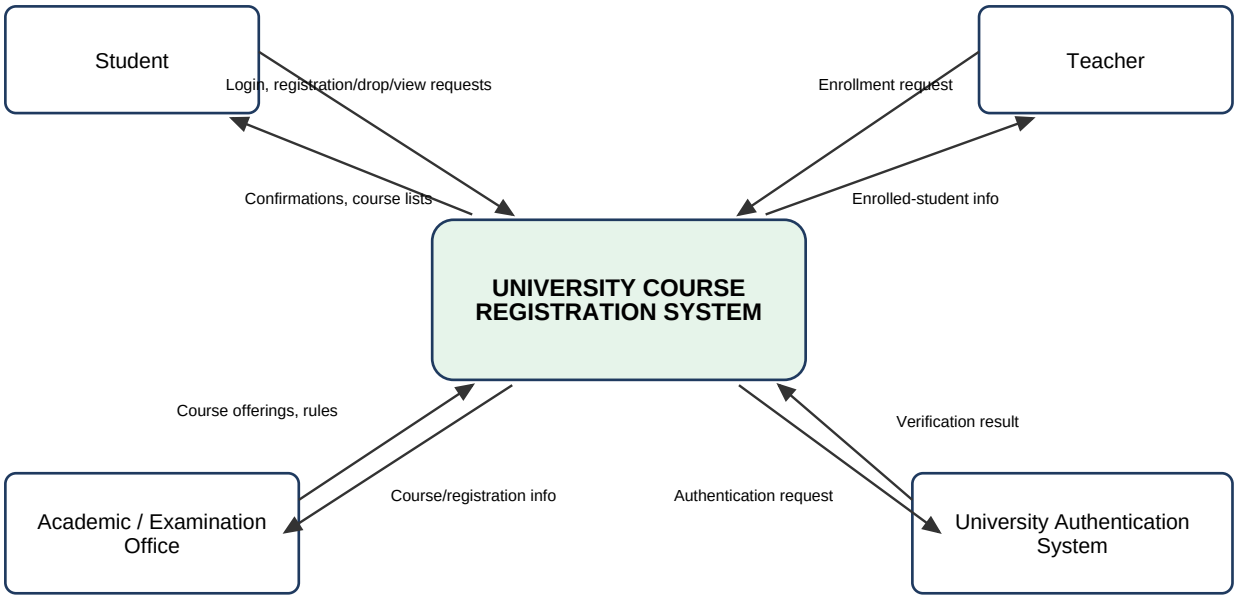



*Figure: Context diagram. Note there are no modules or databases inside the central box.*

## 2.4 Major internal modules

| Module | Responsibility |
|---|---|
| Authentication and Access Control | Handles login/access; talks to external authentication service |
| Course Catalog Management | Maintains/provides available course information |
| Registration Management | Processes registration requests and applies rules |
| Course Drop Management | Processes drop requests |
| Enrollment Management | Maintains/provides enrollment info (e.g. for teachers) |
| Academic Administration | Lets authorised staff manage course offerings and rules |
| Student Records / Registration History | Stores registered-course information and history |

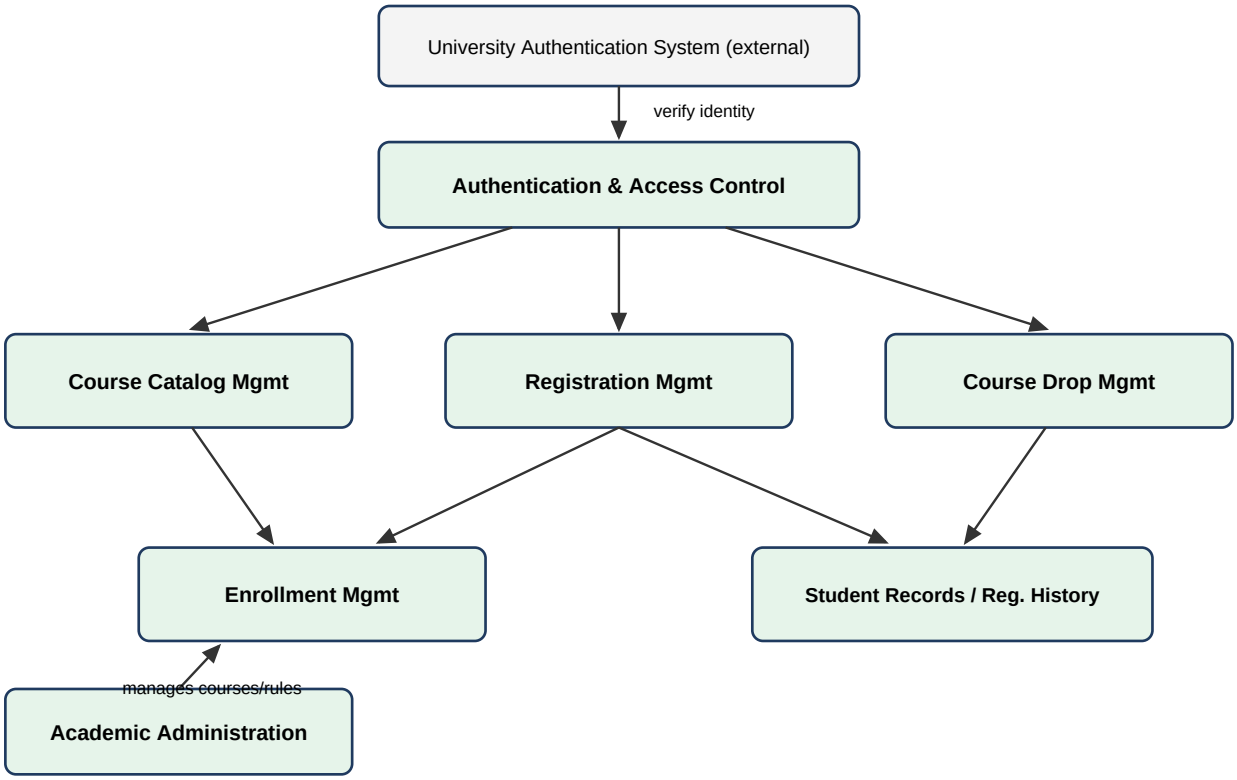



*Figure: System diagram (high-level). Student -> Auth/Catalog/Registration/Drop; Teacher -> Enrollment Mgmt; Academic Office -> Academic Administration.*

## 2.5 Context vs System diagram

| Aspect | Context diagram | System diagram |
|---|---|---|
| Perspective | External/boundary | Internal high-level |
| System shown as | One complete box | Major modules |
| External entities | Shown | May be shown for context |
| Purpose | Boundary and external interactions | Internal organisation |
| Detail | Very high level | More detailed but still conceptual |

## 2.6 Traceability (diagram element -> requirement)

| Req | Context evidence | System evidence |
|---|---|---|
| R1/R8 | Student/Auth System flows | Authentication and Access Control |
| R2 | Student -> System | Course Catalog Management |
| R3 | Student -> System | Registration Management |
| R4 | Student -> System | Course Drop Management |
| R5 | System -> Student | Student Records / Registration History |
| R6 | Teacher -> System | Enrollment Management |
| R7 | Office -> System | Academic Administration |

## 2.7 Viva questions and answers

- **Why is Student outside the boundary?** A person is not a software component; they interact with it.
- **Why is Registration Management inside?** It is functionality we must build to satisfy R3.
- **Why is the Authentication System external?** The scenario says a separate university service verifies identity.

- **Why "Registration Request" not "Data"?** Meaningful labels show what information moves and can be traced to requirements.
- **Why is the database not in the context diagram?** It is internal implementation, not an external entity.
- **Why a separate system diagram?** Context only shows the boundary; the system diagram exposes internal responsibilities.
- **If the university built its own authentication?** The Authentication System would no longer be external; it becomes an internal module and the external flow disappears.
- **If the external service is unavailable?** Users cannot log in; need a defined fallback/error message and availability requirement.

# Part 3: Requirement Sources and the Kano Model

## 3.1 Sources of requirements

| Source | Example | Information obtained |
|---|---|---|
| Students / End users | Students registering | Goals, problems, usability needs |
| Registrar / Academic office | Registration staff | Rules, approval processes, policies |
| Faculty / Instructors | Course instructors | Prerequisites, capacity, constraints |
| IT / System administrators | University IT | Security, infrastructure, integration, constraints |
| Existing system / process | Current workflow | Functions, limitations, workarounds |
| Documents / Policies | Academic regulations | Mandatory rules, deadlines, eligibility |
| Observation | Watch registration | Real behaviour, interruptions, workarounds |
| Interviews / Meetings | Stakeholder discussions | Detailed needs, concerns, exceptions |
| Surveys | Student/staff survey | Patterns across large groups |
| External / comparable systems | Other universities, Google Classroom | Common features, alternatives |

**Your EduTask_pro case** used 5 sources: students, instructors, existing systems (Google Classroom), observation, and informal discussions. **[EXTRA]** To reach the required 8, add: documents/policies, IT/administrators, surveys, comparable apps.

## 3.2 The Kano model (simple explanation)

Kano classifies a feature by how **having or not having** it affects satisfaction. It describes **satisfaction behaviour**, not final priority.

| Category | Meaning | Reaction |
|---|---|---|
| Must-be (M) | Basic expected feature | Not noticed when present, very unhappy when absent |
| One-dimensional (O) | Performance feature | Better performance = more satisfaction (linear) |
| Attractive (A) | Unexpected delighter | Happy when present, no complaint when absent |
| Indifferent (I) | Does not matter | Little difference either way |
| Reverse (R) | Some users prefer it absent | Presence reduces satisfaction |
| Questionable (Q) | Answer pair inconsistent | Cannot classify reliably |

## 3.3 Questionnaire design

For each feature ask two questions with the SAME five options: **I like it / I expect it / I am neutral / I can tolerate it / I dislike it**.

- **Functional:** How would you feel if the system HAD this feature?
- **Dysfunctional:** How would you feel if the system DID NOT HAVE this feature?
- Avoid leading wording ("excellent feature"); keep identification optional; test the form; 15-30 responses are enough for a class demo.

## 3.4 The evaluation matrix (MOST IMPORTANT - memorise)

| Functional \ Dysfunctional | Like | Expect | Neutral | Tolerate | Dislike |
|---|---|---|---|---|---|
| Like | Q | A | A | A | O |
| Expect | R | I | I | I | M |
| Neutral | R | I | I | I | M |
| Tolerate | R | I | I | I | M |
| Dislike | R | R | R | R | Q |

**Shortcuts:** Like + Dislike = O (best). Like + Expect/Neutral/Tolerate = A. Expect/Neutral/Tolerate + Dislike = M. Middle 3x3 block = I. Like+Like or Dislike+Dislike = Q. Anything with Dislike (functional) or Like (dysfunctional) otherwise = R.

**Examples:** Like / Dislike -> O. Like / Neutral -> A. Neutral / Dislike -> M. Neutral / Neutral -> I. Dislike / Like -> R. Like / Like -> Q.

## 3.5 Analysis steps in Google Sheets

- Keep the raw response sheet unchanged. Create a separate classification sheet with columns: Feature, Functional Response, Dysfunctional Response, Kano Category, Notes.
- Apply the evaluation matrix to every pair (52 respondents x 9 features = 468 rows).
- Summarise with COUNTIF per category per feature; find the dominant category (largest count); draw a bar chart.

## 3.6 Your results (n = 52)

| Feature | A | O | M | I | R | Q | Dominant |
|---|---|---|---|---|---|---|---|
| F1 Direct messaging | 22 | 10 | 1 | 15 | 1 | 3 | A |
| F2 Collaborative projects | 13 | 13 | 3 | 14 | 3 | 6 | I |
| F3 Course sub-tabs | 16 | 17 | 1 | 12 | 0 | 6 | O |
| F4 Search bar | 12 | 16 | 7 | 8 | 5 | 4 | O |
| F5 Personal to-do | 12 | 15 | 3 | 12 | 5 | 5 | O |
| F6 Analytics | 17 | 13 | 1 | 14 | 3 | 4 | A |
| F7 Unified assignments view | 14 | 15 | 1 | 16 | 2 | 4 | I |
| F8 Microsoft login | 6 | 4 | 5 | 28 | 5 | 4 | I |
| F9 Peer quiz | 16 | 9 | 2 | 19 | 1 | 5 | I |

## 3.7 Interpretation of findings (write like this)

- **F1 Messaging (A):** delights many users; add if moderation/privacy cost acceptable.
- **F3, F4, F5 (O):** satisfaction grows with implementation quality; prioritise good navigation and search (F4 is low cost, high value).
- **F6 Analytics (A):** delighter, depends on data and design effort.
- **F8 Microsoft login (I):** students mostly indifferent, but IT/security may treat SSO as must-be.
- **F2, F7, F9 (I):** optional/secondary release candidates.

## 3.8 Questionable vs Indifferent (required explanation)

| | Questionable (Q) | Indifferent (I) |
|---|---|---|
| Example (F2) | Respondent 10: Like / Like | Respondent 1: Tolerate / Tolerate |
| Meaning | Pair is logically inconsistent; liking both having and not having | Pair is consistent; person genuinely does not care |
| Implication | Cannot classify; review wording/understanding. Feature is NOT necessarily bad | Feature has little effect on satisfaction |

**Data-quality remark [EXTRA]:** several respondents answered Like/Dislike for almost every feature (possible straight-lining), which can inflate the O counts. Mention this as a limitation.

## 3.9 Reflection: how Kano supports prioritisation (the missing section - write this)

Kano shows how users *feel* about features, helping separate must-haves (M), satisfaction drivers (O), delighters (A) and low-value items (I/R). It supports MoSCoW: M -> Must, O -> Should/Must depending on cost, A ->

Could/Should, I -> Could/Won't, R -> Won't. But Kano alone is not enough: **cost, risk, effort, business value, security, legal requirements and dependencies** must also be considered. For example, security/privacy controls may not appear as A or O in a survey but are still mandatory.

## 3.10 Other considerations table (from lab)

| Feature | Possible Kano | Other considerations |
|---|---|---|
| Registration confirmation | M / O | Business process, usability |
| Peak-period response | O | Performance and infrastructure cost |
| Course recommendations | A | Development effort, recommendation quality |
| Dark mode | I / A | Accessibility, user population |
| Security/privacy controls | May not be A | Risk, policy, compliance, legal |

# Part 4: Requirements Elicitation Techniques (Lab 5)

**Core idea:** Elicitation is evidence-based discovery of needs. It is NOT guessing features. Different techniques answer different questions.

| Category | Techniques | Simple question |
|---|---|---|
| Document-centric | System archaeology, Perspective-based reading, Reuse | What already exists and what can we learn? |
| Observation | Field observation, Apprenticing | What do users actually do? |
| Support | Mind mapping, Workshop, CRC cards, Audio/Video, Use cases, Prototypes | How can we organise, discuss, model or clarify requirements? |

## 4.1 Each technique (definition, how, example, limitation)

| Technique | Definition / what to do | Example | Limitation [EXTRA] |
|---|---|---|---|
| System archaeology | Study an existing/legacy system (code, docs, DB, forms, reports, screens) | Old library system has search, issue, return, fine -> candidate requirements | Shows what exists, not why; may preserve outdated features; details (rules) missing |
| Perspective-based reading | Read the same requirements doc from one stakeholder's viewpoint (developer, tester, security) | "Users can log in": tester asks about wrong password; security asks about lockout | Depends on reviewer skill |
| Reuse | Use requirements from a similar system if still valid | "Students shall view course materials" reused in a new LMS | Must verify relevance/correctness |
| Field observation | Watch users in their real environment; record steps, tools, delays, workarounds | Librarian keeps separate Excel sheet because system lacks a report | Time-consuming; people may behave differently when watched |
| Apprenticing | Learn by doing the work under guidance | Analyst registers a patient and finds repeated data entry | Slow; limited to few tasks |
| Mind mapping | Visual map of requirements, terms, dependencies | Library -> Users, Books, Borrowing -> Search, Issue, Return, Fine | Can become messy; not detailed |
| Workshop | Stakeholders and analysts discuss and agree on requirements | Students want search, teachers want enrollment lists, admin wants seat management | Scheduling, dominant voices |
| CRC cards | Class-Responsibility-Collaboration cards to find classes | Student: register, drop, view timetable; collaborates with Course, Registration, Payment | Design-oriented; needs OO knowledge |
| Audio/Video | Record interviews/work for later review | Recording shows staff check two systems before issuing a book | Privacy, permission needed; much review time |
| Use case / action sequence | Describe actor-system interaction for a goal | Borrow Book: search -> check availability -> request -> confirm | Only shows interactions, not quality needs |
| Prototype | Rough interface so stakeholders can react | Weather wireframe -> users ask hourly forecast and rain alerts | May be mistaken for final design |

## 4.2 Differences (easy exam table)

| Technique | What analyst does | Main question |
|---|---|---|
| System archaeology | Studies existing system | What does the existing system already do? |
| Field observation | Watches users | What do users actually do? |
| Apprenticing | Performs work with guidance | What does the work feel like? |
| Workshop | Discusses with stakeholders | What can stakeholders agree on? |
| Prototype | Shows rough solution | What is missing/unclear when users see it? |
| Use case | Models actor-system interaction | What interaction achieves a goal? |

## 4.3 Submission format (8 points)

1. System selected; 2. Elicitation technique used; 3. Stakeholders involved; 4. Evidence collected; 5. Observations/findings; 6. Candidate requirements; 7. Problems/limitations of technique; 8. Conclusion.

## 4.4 System archaeology model answer: Library Management System

| Existing functionality / evidence | Candidate requirement | Evidence |
|---|---|---|
| Book search screen | System shall allow users to search books | Existing search function |
| Issue form | System shall allow authorised staff to issue books | Existing issue form |
| Return screen | System shall allow staff to record returned books | Return function |
| Fine report | System shall calculate/display overdue fines | Existing report |
| Member table in database | System shall store member records | Sample DB tables |
| Book table (ISBN, title, copies) | System shall maintain book records and copy counts | DB structure |
| Login screen | System shall authenticate staff users | Login form |
| Issue history report | System shall show borrowing history | Report |
| NEW: no email reminders observed | System shall send overdue reminders | Observed limitation |
| NEW: search only by title | System shall allow search by author/ISBN/subject | Limitation |

**Teaching point:** do not invent details (e.g. search by instructor) without evidence. Registration rules like "when can a student drop?" are NOT found by archaeology - use interviews/observation.

## 4.5 Course registration demo: archaeology output

| Existing evidence | Candidate requirement |
|---|---|
| Login | FR-01 The system shall allow students to log in using their credentials |
| Course search | FR-02 ... search for available courses |
| Course selection | FR-03 ... select a course for registration |
| Add course | FR-04 ... add a selected course to registration |
| Drop course | FR-05 ... drop a registered course |
| My timetable | FR-06 ... view registered course timetable |
| Registration history | ... view registration history |

They are **candidate** requirements because they must still be validated with stakeholders. Open questions: what if the course is full? prerequisite problem? same course twice? drop deadline?

## 4.6 Field observation model

Observe one full process (e.g. course registration or library issue): who performs it, what triggers it, steps, information, tools, repeated work, delays, errors, workarounds. Find 5 problems and convert to 5 requirements.

| Observation | Requirement |
|---|---|
| Students check seats in a separate page | System shall show remaining seats at selection |
| Student re-enters same details repeatedly | System shall prefill known student data |
| Registrar uses paper approval | System shall support electronic approval workflow |
| No confirmation after registering | System shall send confirmation after registration |
| Portal hangs at peak time | System shall respond within 2 s for 95% requests at 500 users |

## 4.7 Prototype: weather app

Make 4-5 low-fidelity screens (Home, Weather Details, Forecast, Alerts, Settings/location), show to 2-3 users, record feedback, convert to requirements.

| User feedback | Potential requirement |
| --- | --- |
| I want hourly temperature | System shall display hourly weather |
| I want rain alerts | System shall notify users about rain/severe weather alerts |
| I want to change city | System shall allow selecting/changing location |
| I want Celsius/Fahrenheit | System shall allow unit selection |
| I want a 7-day forecast | System shall display a 7-day forecast |

# Part 5: Use Case Modeling

## 5.1 Concepts

| Term | Meaning |
|---|---|
| Actor | External entity (person, system, device) interacting with the system to achieve a goal. Outside the boundary. |
| Use case | A meaningful user goal (Register for Course), not a button click. |
| System boundary | Rectangle containing use cases; actors are outside. |
| Association | Line between actor and use case. |
| «include» | Base use case ALWAYS needs the included behaviour (mandatory/reusable). Arrow from base to included. |
| «extend» | Optional/conditional behaviour added to base use case. Arrow from extension to base. |
| Generalization | Specialised actor/use case inherits from general one (hollow-triangle arrow). |

**Mindset:** do not ask "what buttons does the software have?" Ask "**who** interacts with the system and what **meaningful goal** are they trying to achieve?"

**Include vs Extend test:** Include: "Is this behaviour required EVERY time?" If yes -> include. Extend: "Can the base finish without it, with the extra behaviour only under a condition?" If yes -> extend.
**Memory trick:** Include arrow points TO the included use case (base -> included). Extend arrow points TO the base (extension -> base).

## 5.2 Course Registration Lab (Tasks 1-8)

**Task 1: System boundary:** System = University Course Registration System. Inside: login handling, course search, view seats, registration, drop, timetable, course/rule management, payment request. Outside: Student, Registrar, Payment Service.

| Actor | Type | Reason for interaction (and why external) |
|---|---|---|
| Student | Human, primary | Registers/drops/views courses; a person, not part of software |
| Registrar | Human, primary | Manages course info and registration rules |
| External Payment Service | External system, secondary | Processes registration fees; separate software |

| Actor | Candidate goals / use cases |
|---|---|
| Student | Log in; Search Courses; View Course Info and Seats; Register for Course; Drop Course; View Timetable; (Pay Registration Fee) |
| Registrar | Manage Courses; Manage Registration Rules |
| Payment Service | Process Payment (secondary actor for Pay Fee) |

**Task 5 relationships:**

- **include:** Register for Course «include» Check Eligibility (prerequisites, seats). Also Register «include» Login/Authenticate (if modelled as separate use case).
- **extend:** Pay Registration Fee «extend» Register for Course (only when the course has a fee).
- **Generalization:** not needed if Student and Registrar are unrelated roles. Say "unnecessary here" rather than inventing one.

## 5.3 Fully dressed use case template (use this structure)

| Field | Content |
|---|---|
| Use Case Name | Register for Course |
| Primary Actor | Student (secondary: Payment Service) |

| Field | Content |
|---|---|
| Goal | Successfully register for an offered course |
| Preconditions | Student is logged in; registration is open; course is offered |
| Main Success Scenario | 1. Student selects a course. 2. System checks registration is open. 3. System verifies prerequisites, credit limit and timetable. 4. System checks seat availability. 5. System displays details and asks confirmation. 6. Student confirms. 7. System saves registration and updates seat count. 8. System displays confirmation. |
| Alternative / Exception Flows | 2a. Registration closed: system informs student, ends. 3a. Condition fails: system shows reason, student selects another course. 4a. No seats: offer waiting list. 6a. Student cancels: no registration created. 7a. Fee required: Pay Fee extension (payment fails -> registration not completed). |
| Postconditions | Registration record saved; seat count decreased; timetable updated; confirmation shown |

## 5.4 SAEMS (Smart Academic and Examination Management System) - full model answer

This is the big "Lab Task 5" scenario. Rule: **not every sentence is a use case**. Internal activities (verify identity, generate confirmation, update timetable, display error, send notification) are system behaviour, not user goals.

### System boundary and actors

| Actor | Type | Role |
|---|---|---|
| Student | Human, primary | Courses, timetable, exam schedule, dues payment |
| Course Instructor | Human, primary | Courses, materials, attendance, marks |
| Department Coordinator | Human, primary | Offerings, instructor assignment, approve special requests |
| Examination Officer | Human, primary | Exam schedules, rooms, publish timetable, results |
| System Administrator | Human, primary | Users, roles, permissions, configuration, reports |
| Identity Authentication Service | External system, secondary | Verifies identity at login; password recovery |
| Bank Payment Gateway | External system, secondary | Processes dues payment, returns result |

**Notification service (email/SMS)**: can be modelled as a secondary actor if external, or treated as an internal system behaviour. State your assumption. **Users with several roles**: model as one person associated with multiple actors (e.g. Instructor who is also a Student) or a generalised actor **University User** with specialised actors; do not duplicate use cases.

### Candidate use cases by actor

| Actor | Use cases |
|---|---|
| Student | Log in; Recover Password; Search Courses; View Course Details; Register for Course; Drop Course; View Timetable; Check Exam Schedule; Pay Dues; Join Waiting List; Request Special Drop |
| Instructor | Log in; View Assigned Courses; View Registered Students; Upload Lecture Materials; Submit Attendance; Enter Internal Marks; Submit Marks |
| Dept Coordinator | Manage Course Offerings; Assign Instructors; Review Special Registration Request (approve/reject); Review Special Drop Request |
| Exam Officer | Create Exam Schedule; Assign Exam Rooms; Publish Exam Timetable; Record Exam Results |
| Sys Admin | Manage User Accounts; Manage Roles and Permissions; Configure System; Generate Reports |

### Relationships

| Type | Relationship | Reason |
|---|---|---|
| include | Register for Course «include» Verify Eligibility (prerequisites, seat availability) | Required every time |
| include | Submit Marks «include» Validate Marks Completeness | Check always runs before submission |
| include | Publish Exam Timetable «include» Check Room/Time Conflicts | Always checked before publication |

| Type | Relationship | Reason |
|---|---|---|
| include | Log in «include» Authenticate User (Identity Service) | Authentication always required (can be shown as association with secondary actor) |
| extend | Join Waiting List «extend» Register for Course | Only if course is full |
| extend | Pay Dues «extend» Register for Course | Only if student has outstanding dues; Bank Gateway attached |
| extend | Request Special Registration «extend» Register for Course | Only if prerequisite missing |
| extend | Submit Special Drop Request «extend» Drop Course | Only after drop deadline |
| extend | Recover Password «extend» Log in | Only if password forgotten |
| generalization | Student, Instructor, Coordinator, Exam Officer, Admin -> University User | All log in; shared behaviour. Optional |

### Things that should NOT be modelled as use cases

- Display error message, generate confirmation, update timetable, update seat count, validate marks input errors (internal system responses).
- Send email/SMS notifications (system-triggered, not a user goal).
- "Click register" or other UI actions.
- Allow instructor to correct incomplete marks: part of Enter/Submit Marks flow (alternative flow), not a separate use case.

### Fully dressed use case example: Submit Internal Marks

| Field | Content |
|---|---|
| Name | Submit Internal Assessment Marks |
| Primary actor | Course Instructor |
| Preconditions | Instructor logged in; assigned to the course; marks entry open |
| Main success | 1. Instructor selects course. 2. Enters marks for all assessment components. 3. Submits. 4. System validates completeness. 5. System saves marks. 6. System confirms submission (and may notify). |
| Alternative | 4a. Incomplete marks: system shows error and highlights missing components; instructor corrects and resubmits. |
| Postconditions | Marks stored and locked/recorded; submission confirmation shown |

## 5.5 Reflection questions - model answers

- **Actor vs user:** an actor is a ROLE (can be a person, system or device); a user is a specific person. One user can play several roles.
- **Why actors outside boundary:** they are not part of the software being built; the boundary separates what we build from who/what uses it.
- **Register for Course vs Click Register Button:** the first is a user goal with value; the second is a UI action.
- **include vs extend:** include = mandatory/reusable; extend = optional/conditional.
- **Mistake if every function is a use case:** diagram becomes cluttered, loses goal focus, mixes UI/internal steps with user goals.
- **Use case diagram vs fully dressed:** diagram gives a high-level picture of actors and goals; fully dressed use case describes steps, conditions and exceptions.
- **What diagram leaves out:** step-by-step flows, preconditions, postconditions, business rules, data, UI details.

## 5.6 Review checklist

Boundary clear; actors outside; use cases inside; each use case a meaningful goal; each actor's role explained; no UI actions; includes truly mandatory; extends truly conditional; correct directions/labels; readable; no duplicates.

# Part 6: Data Flow Diagram (DFD)

## 6.1 Concepts

| Symbol | Meaning | Naming rule |
|---|---|---|
| Rectangle / external entity | Source or sink of data outside system | Noun: Student, Academic Administration |
| Circle / rounded box = process | Transforms data | Verb + noun, numbered 1.0, 2.0: Validate Registration |
| Open-ended rectangle = data store | Data at rest | D1, D2: Student Records |
| Arrow = data flow | Data moving | NOUN labels (data), not actions: "Login Credentials", not "Login" |

**DFD rules:** (1) Every flow is labelled with data. (2) Flows connect to a process on at least one side: **no entity-to-entity, entity-to-store, or store-to-store flows**. (3) No process without input and output. (4) Data stores are accessed only through processes. (5) Level 1 must balance with the context diagram (same external inputs/outputs). (6) Flows are data, NOT commands like "check" or "approve".

## 6.2 Level 0 (context) and Level 1

Level 0 = one process (Course Registration System) with Student and Academic Administration. Level 1 decomposes into logical processes. **Your task: decompose, identify data stores, connect with labelled flows.**

## 6.3 Processes, stores, entities

| Item | Items |
|---|---|
| External entities | Student; Academic Administration Staff |
| Processes | 1.0 Authenticate Student; 2.0 Provide Course List; 3.0 Validate Registration; 4.0 Record Registration and Confirm; 5.0 Generate Registration Report |
| Data stores | D1 Student Records; D2 Course Records (prerequisites, seats); D3 Registration Records |

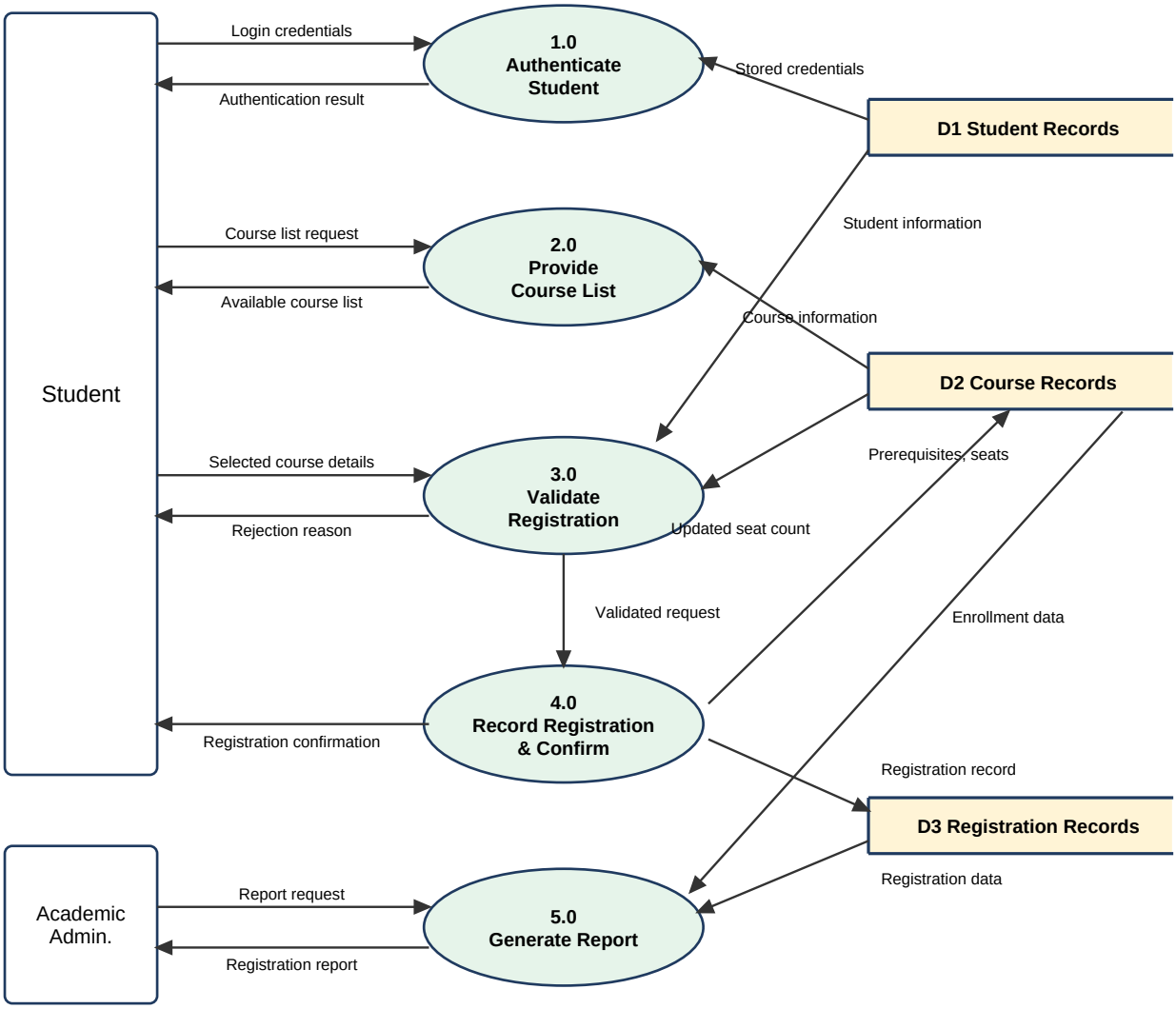



*Figure: Level 1 DFD (schematic). If asked to redraw, draw bigger and spread flows; clarity matters more than exact positions.*

## 6.4 Data flow list (write this table in the exam)

| From | To | Data flow (label) |
|---|---|---|
| Student | 1.0 | Login credentials |
| D1 | 1.0 | Stored student credentials |
| 1.0 | Student | Authentication result |
| Student | 2.0 | Course list request |
| D2 | 2.0 | Course information |
| 2.0 | Student | Available course list |
| Student | 3.0 | Selected course details (registration request) |
| D1 | 3.0 | Student information |
| D2 | 3.0 | Prerequisites and available seats |
| 3.0 | Student | Rejection reason (unmet prerequisites / no seats) |
| 3.0 | 4.0 | Validated registration request |
| 4.0 | D3 | Registration record |
| 4.0 | D2 | Updated seat count (if assumed) |
| 4.0 | Student | Registration confirmation |
| Academic Administration | 5.0 | Registration report request |

| From | To | Data flow (label) |
|---|---|---|
| D3 | 5.0 | Registration data |
| D2 | 5.0 | Course enrollment data |
| 5.0 | Academic Administration | Registration report (registrations, enrollment statistics) |

## 6.5 Common mistakes

- Labelling arrows with actions ("Register", "Check seats") instead of data.
- Drawing student directly to data store.
- Process with only outputs (miracle) or only inputs (black hole).
- Forgetting rejection path or the report process.
- Showing implementation details (SQL, screens).

## 6.6 DFD vs Flowchart vs Context

DFD shows **data movement** (what data goes where) with no control logic or sequence. A flowchart/activity diagram shows **control flow** (steps and decisions). Context diagram is Level 0 DFD.

# Part 7: Software Requirements Specification (SRS)

## 7.1 What an SRS is

An SRS is **not simply a description** of a system; it is an **agreement** about what the system must do and the constraints under which it must operate. The reference standard is **ISO/IEC/IEEE 29148**.

## 7.2 SRS structure (practical, standards-aligned)

| Section | Contents |
|---|---|
| 1. Introduction | Purpose, scope, audience, definitions/acronyms, references, overview |
| 2. Overall description | Product perspective/boundaries, product functions, user classes, operating environment, assumptions and dependencies, constraints |
| 3. Functional requirements | Functions, inputs, processing, outputs, business rules, pre/postconditions, error handling, role-specific behaviour |
| 4. Non-functional requirements | Performance, security/privacy, usability/accessibility, reliability/availability, scalability/maintainability, compatibility/portability, legal/regulatory |
| 5. External interface requirements | User interfaces, software interfaces/APIs, hardware, communication protocols |
| 6. Data and business rules | Data entities and fields, validation rules, retention/deletion, policies and eligibility rules |
| 7. Verification, traceability, document control | Unique IDs, acceptance criteria/verification, traceability to sources, version/date/author/revision history, open issues, approval |

Not every heading is universally mandatory (templates combine sections), but the requirements themselves must be sufficiently complete.

## 7.3 Writing correct requirements (the most important skill)

| Weak | Improved |
|---|---|
| The system should allow students to register for courses easily. | FR-REG-001: The system shall allow an authenticated student to submit a course registration request for an offered course, provided that the student meets the course prerequisites, has no timetable conflict, and has sufficient credit hours remaining. |
| The system shall be fast. | NFR-PER-001: Under a load of 500 concurrent users, 95% of course-search requests shall complete within 2 seconds, measured at the application interface. |

The improved FR states **actor, action and conditions**. The improved NFR gives **threshold, workload, measurement point and percentile** so it is verifiable. Targets must be agreed with stakeholders.

## 7.4 Characteristics of a good requirement (checklist)

| Characteristic | Meaning |
|---|---|
| Correct | Reflects a genuine stakeholder need |
| Unambiguous | Only one reasonable interpretation |
| Complete | Enough information to understand behaviour |
| Consistent | Does not conflict with another requirement |
| Feasible | Can be implemented within constraints |
| Verifiable | Can be checked by test, inspection, analysis or demonstration |
| Necessary | Contributes to a required capability/constraint |
| Traceable | Linked to source and downstream design/test |
| Singular | One principal requirement, not combined obligations |

Avoid vague words: easy, fast, efficient, appropriate, normally, as soon as possible (unless defined objectively).
**Mnemonic:** CUC-CF-VNTS (Correct, Unambiguous, Complete, Consistent, Feasible, Verifiable, Necessary, Traceable, Singular).

## 7.5 Requirement specification table

| ID | Requirement | Priority | Verification |
|---|---|---|---|
| FR-001 | The system shall allow students to search offered courses by course code or title. | High | Test |
| FR-002 | The system shall display available seats for each offered course. | High | Test |
| NFR-001 | 95% of search requests shall complete within 2 seconds under the agreed workload. | High | Performance test |
| SEC-001 | The system shall deny access to registration functions for unauthenticated users. | High | Security test |

Optional columns: source, rationale, dependencies, status, acceptance criteria.

## 7.6 [EXTRA] Mini SRS for the Course Registration System (use as a template)

- **1 Introduction:** Purpose - specify requirements of the University Course Registration System. Scope - online registration, drop, timetable, reports. Audience - developers, testers, registrar, IT.
- **2 Overall description:** Users: students, teachers, registrar, IT admin. Environment: web, modern browsers. Dependency: University Authentication System. Constraint: prerequisite and credit-hour rules.
- **3 Functional:** FR-001 authentication; FR-002 search; FR-003 show seats; FR-REG-001 registration with conditions; FR-005 reject with reason; FR-006 confirmation; FR-007 drop before deadline; FR-008 teacher views enrolled students; FR-009 registrar manages courses/rules; FR-010 reports.
- **4 NFR:** Performance (2 s at 500 users), Security (lockout after 3 failures, TLS), Availability (99.5% in registration week), Usability, Scalability (5,000 concurrent users).
- **5 Interfaces:** Web UI; interface to Authentication System (protocol, e.g. HTTPS/SSO).
- **6 Data/rules:** entities Student, Course, Registration; max credit hours; prerequisites; drop deadline.
- **7 Traceability:** each requirement has ID, source (e.g. Registrar interview), verification method; version history.

# Part 8: UML Activity Diagram and State Machine Diagram

## 8.1 Activity diagram basics

| Element | Notation | Use |
|---|---|---|
| Initial node | Filled black circle | Start |
| Final node | Circle with inner black dot (bull's-eye) | End |
| Action | Rounded rectangle | A step, e.g. Verify prerequisites |
| Control flow | Arrow | Order of steps |
| Decision | Diamond with guards [condition] | Branch (one in, many out) |
| Merge | Diamond | Joins alternate paths (many in, one out) |
| Fork/Join | Thick bar | Parallel activities |
| Swimlane [EXTRA] | Vertical/horizontal partitions | Who performs the action (Student / System) |

**Rules:** Every decision needs **guards** on every outgoing arrow (e.g. [valid], [invalid]) and the guards must cover all cases. Use a merge when branches rejoin. Loopbacks are arrows returning to an earlier action or merge.

## 8.2 Activity diagram for the registration workflow (build step-by-step)

Draw swimlanes: **Student** and **System**. Follow this sequence:

| # | Node | Guards / notes |
|---|---|---|
| 1 | Initial node -> Enter credentials (Student) | |
| 2 | Validate credentials (System) -> Decision D1 | [valid] continue; [invalid] -> show error -> check attempts |
| 3 | Decision D2: failed attempts = 3? | [no] -> loop back to Enter credentials; [yes] -> lock account temporarily -> wait for lock expiry -> back to Enter credentials (or Final node) |
| 4 | View offered courses -> Select course | Merge point M1 here (all 'select another course' loops return) |
| 5 | Decision D3: registration open? | [closed] -> inform student -> Final node; [open] -> continue |
| 6 | Check prerequisites, credit capacity, timetable conflict | Decision D4: all satisfied? |
| 7 | D4 [no] | Display reason for rejection -> loop to M1 (select another course) |
| 8 | D4 [yes] -> Decision D5: seats available? | |
| 9 | D5 [no] -> Decision D6: waiting list enabled and not full? | [yes] join waiting list (-> seat later offered, accept in time -> register, else offer expires -> next student); [no] -> choose another course -> M1 |
| 10 | D5 [yes] -> Display course details -> ask confirmation -> Decision D7 | [confirm] save registration, update seat count, show confirmation; [cancel] no registration -> M1 |
| 11 | After registration: Decision D8 (choose action) | View timetable; Drop course; Register another (-> M1); Log out |
| 12 | Drop course: Decision D9 deadline not passed? | [yes] update registration record, release seat to eligible student; [no] reject with message |
| 13 | Log out -> Final node | |

**Waiting list sub-flow:** Seat becomes available -> notify next eligible student -> start limited-time offer -> Decision [accepted within period]: register student; [expired]: offer to next eligible student (loop).

### Swimlane summary (easy to draw)

| Student lane | System lane |
|---|---|
| Enter credentials; select course; choose waiting list; confirm/cancel; view timetable; request drop; log out | Validate credentials; lock account; show course list; check registration open; check eligibility; check seats; manage waiting list and offers; save registration; update seats; display messages; process drop |

## 8.3 State machine diagram basics

- **State:** a condition of ONE object during its life (rounded rectangle). **Transition:** arrow labelled **event [guard] / action**.
- **Initial state** (black dot), **final state** (bull's-eye). Events trigger transitions; guards are boolean conditions; actions are executed during the transition.
- Difference from activity diagram: activity = *workflow of steps* across actors; state = *life cycle of one object* (here, a Registration Request).

## 8.4 State machine for Registration Request

| State | Description |
|---|---|
| Initiated / Selected | Student has selected a course |
| Validating (Eligibility Check) | System checks if registration open and prerequisites, credits, timetable |
| Rejected | Final: failed eligibility or registration closed |
| Checking Seats | Eligibility passed; seat availability check |
| Waitlisted | No seat; student on waiting list |
| Offered | Seat offered to waitlisted student; timer running |
| Pending Confirmation | Seat available; awaiting student confirm/cancel |
| Cancelled | Final: student cancelled |
| Registered | Registration saved; seat count updated |
| Dropped | Final: student dropped course (deadline respected) |
| Offer Expired | Offer timed out; seat goes to next student; (request returns to Waitlisted or ends) |

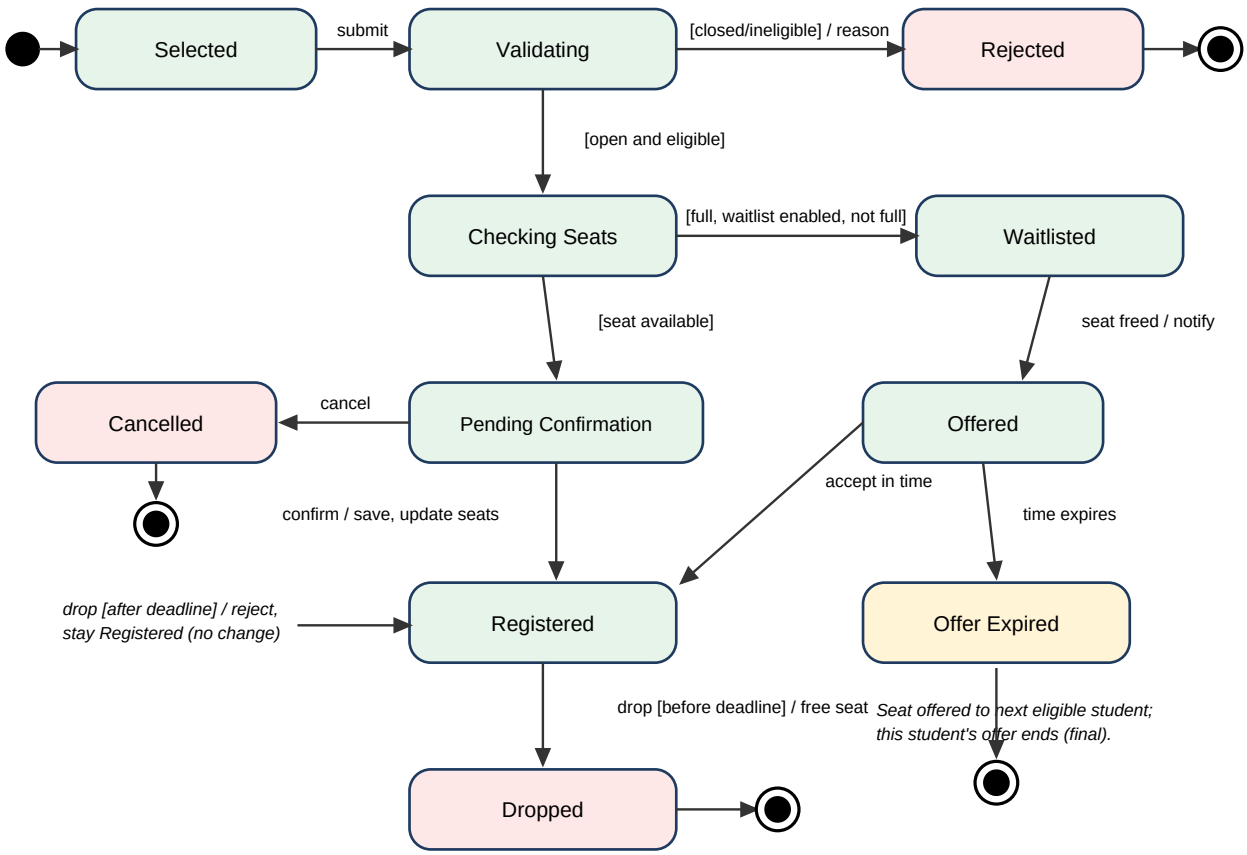



*Figure: State machine (schematic). Bull-eye circles are final states: Rejected, Cancelled, Dropped, and the Offer Expired end of the offer.*

**Transition table (write this in the exam if drawing is short of time)**

| From | Event | Guard | Action | To |
|---|---|---|---|---|
| (initial) | Select course | | Create request | Selected |
| Selected | Submit | | Run checks | Validating |
| Validating | Checks done | Registration closed OR condition fails | Show reason | Rejected |
| Validating | Checks done | Open AND eligible | Check seats | Checking Seats |
| Checking Seats | Seat check | Seat available | Show details | Pending Confirmation |
| Checking Seats | Seat check | No seat AND waitlist enabled AND not full | Add to list | Waitlisted |
| Checking Seats | Seat check | No seat AND no waitlist | Show message | Rejected / choose another |
| Pending Confirmation | Student confirms | | Save, update seats, confirm | Registered |
| Pending Confirmation | Student cancels | | No registration created | Cancelled |
| Waitlisted | Seat freed | Next eligible | Notify, start timer | Offered |
| Offered | Student accepts | Within period | Register student | Registered |
| Offered | Timeout | Period elapsed | Offer to next student | Offer Expired |
| Registered | Drop request | Before deadline | Update record, release seat | Dropped |
| Registered | Drop request | After deadline | Reject, show message | Registered (no change) |

## 8.5 Login lockout state (small state machine) [EXTRA]

Account states: **Active -> (invalid login, count<3) -> Active**; **Active -> (3rd consecutive failed login) -> Locked / start timer**; **Locked -> (lock period expires) -> Active**; **Active -> (valid login) -> Logged in**. Good to mention if the exam asks about login retries.

# Part 9: Final Cheat Sheet and Likely Exam Questions

## 9.1 One-page formulas

| Topic | Remember |
|---|---|
| Good requirement | Shall + action + condition + measurable value + verification |
| Vague words to avoid | fast, easy, secure, user-friendly, efficient, soon, appropriate, normally |
| FR vs NFR | FR = what it does; NFR = how well (performance, security, usability, availability, scalability) |
| MoSCoW | Must, Should, Could, Won't (this time) |
| Kano | O = Like+Dislike; A = Like+(Expect/Neutral/Tolerate); M = (Expect/Neutral/Tolerate)+Dislike; I = middle 3x3; R = Dislike functional or Like dysfunctional; Q = Like/Like or Dislike/Dislike |
| Context diagram | 1 system box, external entities, labelled directional flows, no internals |
| System diagram | 5-8 internal modules and relationships, no DB columns/code |
| Use case | Actors outside, use cases inside; include = always; extend = sometimes |
| DFD | Processes numbered, stores D1.., flows are data nouns, no entity-store direct flow |
| Activity diagram | Initial, actions, decision with guards, merge, loops, final |
| State diagram | Event [guard] / action; one object's life |
| SRS | Agreement; IEEE 29148; unique IDs; verifiable and traceable |

## 9.2 Short-question bank with model answers

- **Define RE.** Systematic process of eliciting, analysing, specifying, validating, documenting and managing requirements.
- **Why do projects fail due to requirements?** Missing, ambiguous, conflicting and changing requirements lead to rework, cost and delay.
- **Difference between FR and NFR with examples.** FR: student can drop course. NFR: 95% of searches within 2 s.
- **What makes a requirement testable?** Measurable criterion, workload and measurement method.
- **What is Kano? Explain O, A, M.** See 3.2; give example: search bar O, messaging A, login M.
- **Why Kano result is not final priority?** Cost, risk, legal, dependencies matter too.
- **Questionable vs Indifferent?** Q = inconsistent answers; I = consistent genuine neutrality.
- **System archaeology?** Studying an existing system's functions/docs to discover candidate requirements.
- **Why "candidate" requirements?** Must be validated with stakeholders before acceptance.
- **Field observation vs apprenticing?** Watching users vs doing the work yourself under guidance.
- **Prototype purpose?** Concrete artefact to elicit missing/unclear requirements, not a working app.
- **Actor vs use case?** Actor = who; use case = goal.
- **Include vs extend?** Mandatory vs conditional.
- **DFD vs flowchart?** Data movement vs control flow.
- **Activity vs state diagram?** Process workflow vs object life cycle.
- **Context vs Level 1 DFD?** One process vs decomposed processes and data stores.

## 9.3 Likely practical questions and how to answer

| If the exam asks... | Do this |
|---|---|
| Draw context/system diagram for a given scenario (library, hospital, e-commerce) | List entities and flows first; draw one central box; then 5-8 modules |
| Write FR/NFR for a system | 10 FR in shall format, 5 NFR with numbers; use IDs |
| Find problems in given requirements | Name problem type (ambiguous, incomplete, untestable, conflicting) + rewrite |

| If the exam asks... | Do this |
|---|---|
| Kano classification of response pairs | Use the matrix; say which category and why |
| Apply an elicitation technique | Use the 8-point format; give evidence -> candidate requirements |
| Use case diagram + description | Actors, goals, include/extend with justification, one fully dressed use case |
| Level 1 DFD | 3-5 processes, 3 stores, labelled data flows, check balancing |
| Activity/state diagram | Guards on every decision; loops; final nodes; transition labels event[guard]/action |
| SRS section | Use the 7-part structure; write measurable requirements |

## 9.4 Last-minute reminders

- Say your **assumptions** when the scenario is unclear.
- Label everything and keep diagrams uncluttered.
- Use consistent names across diagrams (same actors, same modules).
- Time plan: read (5 min), plan assumptions (5 min), answer easy marks first, leave drawing refinement for last.

**Good luck!** If you remember only one thing: **clear, measurable, testable requirements; correct boundaries; labelled flows; guards on every decision.**